Investigation & Architecture Map:

SongNodes DJ Track Transition Graph & Curated Library Platform

A deterministic, multi-signal architectural investigation mapping end-to-end data flows, storage hubs, user journeys, operational gaps, and sister-project synchronization across SongNodes and musicLibrary.

Commit: `7429821ca1` **Design System:** v1.4.0 (Tokens Only) **Generated:** 2026-11-07T09:05:00+11:00

Locale: en-AU (Australia/Melbourne)

Section 1: **Executive Summary & Headline Metrics**

SongNodes captures, enriches, and visualizes proven sequential track transitions (`position N -> position N+1`) from real DJ mixes. The architecture enforces strict scope boundaries (G1 scope guard, G2 route registry, G3 REST router registry) and operates a local-first metadata enrichment waterfall alongside its sister repository `musicLibrary`.

12
CATALOGUED VIEWS
Registered static and dynamic feature routes

10
PRODUCTION PAGES
Fully tested, live operational tools

2
NON-FINAL STUBS
Superseded prototypes (PIXI Graph) and fallback stubs

8
STORAGE HUBS
Postgres Medallion, Memgraph, Redis, SQLite, Filesystem

26
DATA EDGES
Traced network, SQL, IPC, and filesystem contracts

5
CRITICAL GAPS
Missing feedback loops and dual-storage disconnects

Section 2: **Page Catalog & Production Classification Audit**

Every operational UI entry point is audited using dual independent signals (route registry + live API wiring + automated test coverage).

PAGE & ROUTE	MODULE	CLASSIFICATION	OPERATIONAL FUNCTION	CODE EVIDENCE & VERIFI
SongNodes /	home	• Production	System launcher, high-level operational overview, and real-time statistics console.	Static entry route registered in regis Design System SN helpers; live stat frontend/src/pages/HomePag frontend/src/routes/regist
Music Library /library	library	• Production	Curated DJ track catalog, crate filtering, energy tier exploration, and track-to-target scraper promotion.	Dynamic feature route discovered v musicLibrary/route.tsx:9; verified liv #259. frontend/src/features/musi frontend/src/features/musi
Set Builder /sets	sets	• Production	Harmonic wave-shaped DJ set sequencer with Camelot transition analysis, energy curve tracking, and multi-platform export.	Production operational tool; dual pe PostgreSQL library.saved_sets); har frontend/src/features/musi 681 frontend/src/features/musi
Pathfinder /paths	paths	• Production	Graph-based transition pathfinder calculating harmonic, energy-aligned stepping stones between distant tracks.	Restored in package R2 (#177) and Memgraph pathfinding Cypher quer frontend/src/features/path frontend/src/features/path

PAGE & ROUTE	MODULE	CLASSIFICATION	OPERATIONAL FUNCTION	CODE EVIDENCE & VERIFI
Camelot Matrix `/harmony`	harmony	• Production	Interactive Camelot wheel and harmonic modulation matrix displaying musical compatibility and energy shift potential.	Consolidated into single production official SN.camelot and SN.compat `frontend/src/features/came` `frontend/src/features/came`

Scrape Targets
`/sources/targets`
	sources	• Production	Priority queue and management console for artist and track search targets dispatched to unified scrapers.	Production target dispatch tool; bac
				duplicate target insertion and coord
				`frontend/src/pages/ModuleP`
				`frontend/src/components/Ta`

Tracklist Import `/sources/import`	sources	• Production	AI-assisted tracklist ingestion, OCR text parsing, and sequential playlist bronze layer loader.	Feature route wired in P3 (#185); en with no gaps) and loads into bronze `frontend/src/features/aiSe` `frontend/src/features/aiSe`

Pipeline Telemetry
`/pipeline`
	pipeline	• Production	Real-time scraper telemetry, Celery queue drain rates, Medallion stage progression, and job dead-letter revival.	Core operational health console; mo
				data layer consistency across Meda
				`frontend/src/pages/Scraper`
				`frontend/src/features/pipe`
				`250`

System Settings
`/settings`
	services	• Production	Credential store, OAuth streaming token manager (TIDAL & Spotify), API provider configuration, and display preferences.	Production administrative and prefe
				credentials without committing raw
				`frontend/src/pages/Setting`
				`frontend/src/routes/regist`

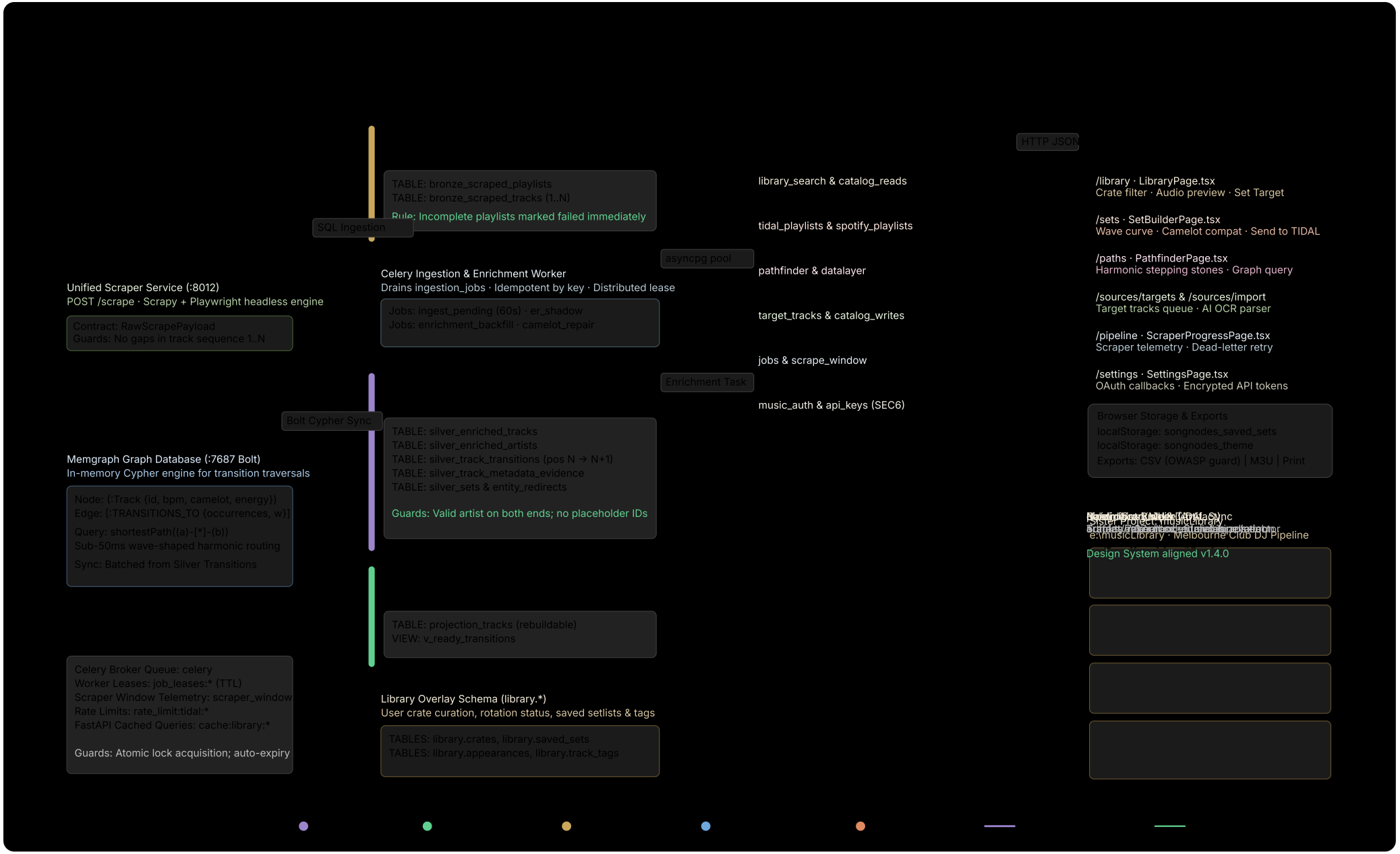
PAGE & ROUTE	MODULE	CLASSIFICATION	OPERATIONAL FUNCTION	CODE EVIDENCE & VERIFI
Page Not Found *	home	● Production	Honest empty/error boundary state displayed when a requested URL is unregistered.	Canonical error boundary adhering never mock data); registered in regi `frontend/src/pages/ModuleF` `frontend/src/routes/regist`
Transition Graph `/graph`	graph	▲ Non-Final	Legacy 2D transition graph visualiser (PIXI/Canvas prototype slated for Sigma.js replacement in Milestone 6).	Classified as superseded prototype mandates replacing legacy PIXI ren conformance (pending exemption ir `frontend/src/pages/GraphPa` `docs/IMPLEMENTATION_PLAN.m`
Planned Module `/paths (stub)`	paths	▲ Non-Final	Static fallback placeholder that is dynamically superseded by the real Pathfinder feature route.	Classified as redirect/stub. Provided replaced dynamically at runtime by `frontend/src/pages/ModuleF` `frontend/src/routes/regist`

Section 3: **System Architecture & Data Flow Topology**

Complete end-to-end data flow topology illustrating External Spiders, the PostgreSQL Medallion Core (Bronze, Silver, Gold, Library), Memgraph In-Memory Graph, Redis Broker, FastAPI REST API, React Frontend, and Sister Project (`musicLibrary`).

Figure 1: Full System Architecture & Data Flow Contracts

ViewBox 1800×1100 · Standalone SVG



Section 4: **Persona Operational Journeys & Step Cards**

Horizontal swimlane execution flow mapping the 5 primary user roles across page progressions, transition triggers, data handoff schemas, and failure modes.

Figure 2: Persona Operational Flow Swimlanes

ViewBox 1800×1100 · Standalone SVG

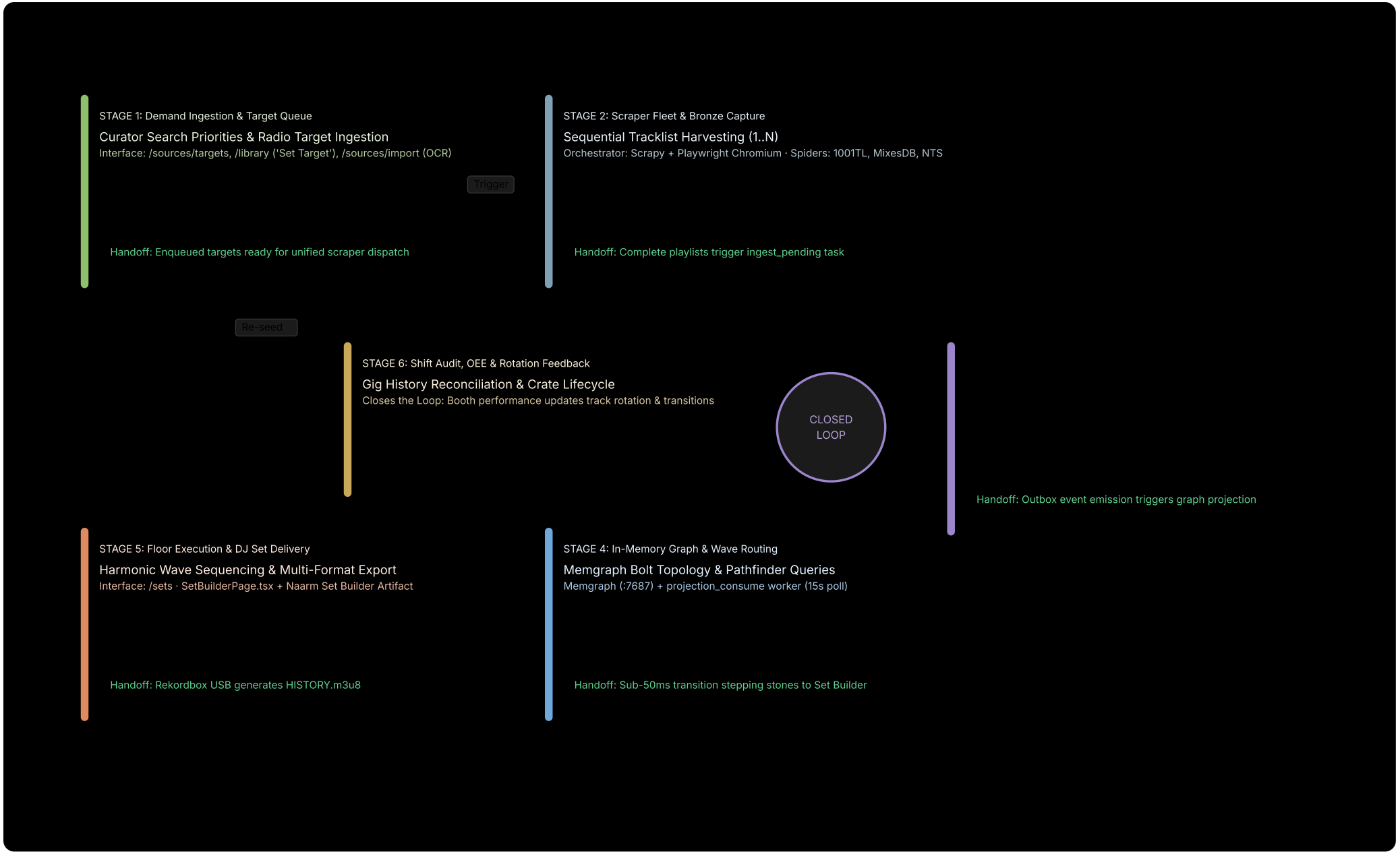


Section 5: **Production Execution Closed-Loop Architecture**

The complete 6-stage industrial DJ curation cycle: Demand Ingestion → Scheduling → Normalisation → Graph Projection → Floor Execution → Reconciliation Feedback.

Figure 3: Production Execution Closed-Loop Cycle

ViewBox 1800×1100 · Standalone SVG



Section 6: **Storage Hubs & Physical Topologies**

SongNodes coordinates 8 storage hubs, balancing ACID relational guarantees in PostgreSQL with sub-50ms graph queries in Memgraph and resilient local offline storage in the browser and SQLite.

PostgreSQL Bronze Ingestion Store `(postgres_bronze)`

relational_db · Docker Volume pgdata (PostgreSQL port 5432)

Raw, immutable capture of external scraped playlists and track items.

Lock / Concurrency:
Transaction-scoped MVCC; unique constraints on (source, source_playlist_id)
Authoritative: **Yes, for un-normalized raw source payloads.**

PostgreSQL Silver Normalized Store `(postgres_silver)`

relational_db · Docker Volume pgdata (PostgreSQL port 5432)

Deduplicated, enriched tracks, verified sequential transitions (1..N), and provider consensus evidence.

Lock / Concurrency:
Row-level locks during Celery enrichment worker drain; idempotent by job_id
Authoritative: **Yes, authoritative single source of truth for all verified track pairings and metadata consensus.**

PostgreSQL Gold Projection Store `(postgres_gold_projection)`

relational_db · Docker Volume pgdata (PostgreSQL port 5432)

Denormalized, read-optimized projection of tracks and transitions for fast API queries.

Lock / Concurrency:
Maintained by projection_consume Celery worker; rebuildable from silver.
Authoritative: **No, disposable derived projection.**

Memgraph Graph Database `(memgraph_gold_graph)`

graph_db · Docker container songnodes-memgraph (Bolt port 7687)

In-memory graph topology enabling sub-50ms Cypher queries for harmonic pathfinding and transition traversals.

Lock / Concurrency:
Internal Memgraph Bolt transaction concurrency
Authoritative: **No, projected from PostgreSQL silver transitions.**

Redis Broker, Lease & Telemetry Store `(redis_queue_cache)`

in_memory_kv · Docker container songnodes-redis (port 6379)

Celery job queue broker, worker lease management, rate-limiting counters, and real-time scraper window telemetry.

Lock / Concurrency:
Redis single-threaded atomic operations; TTL-backed leases

Authoritative: **No, transient operational state.**

musicLibrary Curated SQLite Database `(musiclibrary_sqlite)`

sqlite_db · e:\musicLibrary\db\dj_library.sqlite

Local offline curated DJ database holding crates, appearances, rotation history, and setlists.

Lock / Concurrency:
File-based SQLite locks; single-writer batch rebuild via pipeline/run_weekly.py db

Authoritative: **Yes, authoritative for DJ crate assignments and High/Standard/Archive rotation lifecycle.**

Filesystem Ledger & Export Volumes `(filesystem_ledgers_outputs)`

filesystem · e:\musicLibrary\ledger\, e:\musicLibrary\outputs\, frontend\public\data\

Immutable weekly ledger snapshots, crate CSVs, Rekordbox XML exports, and static sync files.

Lock / Concurrency:
Atomic file rename/write operations

Authoritative: **Yes, ledger is the permanent audit trail for weekly rotation calculations.**

Client Browser LocalStorage & SessionStorage `(client_browser_storage)`

browser_kv · Browser Origin LocalStorage / SessionStorage

Offline set outbox, active UI theme preference ('dark'|'light'|'booth'), and transient OAuth state.

Lock / Concurrency:
Single-thread per tab; storage events across same-origin tabs

Authoritative: **Yes, for un-synced offline draft setlists.**

Section 7: **Inter-Page & Inter-Process Protocols**

1. Dual-Persistence & Offline Outbox Protocol

`SetBuilderPage.tsx` commits changes synchronously to browser LocalStorage (`songnodes_saved_sets`) on every drag-and-drop or slot edit. Asynchronously, mutations are posted to `POST /api/v1/library/sets`. If the network or backend is offline, the local draft persists indefinitely, alerting the user with an honest status banner.

2. Streaming OAuth Popup Handshake (postMessage)

Music streaming authentication for TIDAL and Spotify operates via secure popup windows. A cryptographic nonce state is stored in `sessionStorage`. Upon user approval, `OAuthCallback.tsx` validates the nonce and securely transmits the token to the parent opener using `window.opener.postMessage({type: 'OAUTH_SUCCESS', token: ...}, window.location.origin)`. Server credentials are encrypted on disk via SEC6; no plaintext secrets reside in browser storage.

3. OWASP CSV Formula Injection Guard (DR-033)

All CSV exports across SongNodes and musicLibrary pass through the OWASP formula injection guard: any string cell whose leading character is `=`, `+`, `-`, `@`, tab, or carriage return is prefixed with a single apostrophe (`'`). This prevents spreadsheet execution while preserving raw numeric sorting on BPM and energy values.

Section 8: **Personas & Operational User Journeys**

Detailed breakdown of user objectives, step progressions, transition triggers, and fallbacks. Select a persona below:

Performing DJ / Live Selector

Primary Persona 1 of 5

Objective: Design harmonically locked, energy-curved DJ setlists with proven track pairings, exportable to USB / Rekordbox / TIDAL.

Morning Routine: Opens /sets or /library on Friday afternoon before weekend gigs; reviews freshly scraped high-rotation crates; constructs 2-hour wave curve.

Step-by-Step Progression & Data Handoffs:

Step 1 · / (Home)

Review latest library track count and freshness stamp

Trigger: Initial login / load

Step 2 · /library

Filter by 'High Rotation' and energy tier 'Building (5-7)'

Trigger: Click 'Explore Library' tile

Step 3 · /library

Audition 30s preview snippets; click 'Add to Set'

Trigger: Audio player preview / Add action

Step 4 · /sets

Drag track into Slot N; inspect Camelot compatibility badge (SN.compat)

Trigger: Navigate to Set Builder

Step 5 · /paths

Stuck bridging between 8A (124 BPM) and 3B (128 BPM); query pathfinder for 2-step harmonic bridge

Trigger: Click 'Find Bridge' on slot

Step 6 · /sets

Accept suggested bridge; click 'Export Set' → choose Rekordbox XML and TIDAL playlist

Trigger: Click 'Export' button

Failure Modes & Resilient Fallbacks:

- **Backend API unreachable:** Dual persistence saves draft set directly to browser LocalStorage (songnodes_saved_sets); shows offline warning toast.
- **Missing audio preview snippet:** Displays honest en dash (sn-none) with no audio playback; does not fabricate audio.
- **Track missing local audio file path:** Rekordbox XML export skips unlinked tracks and alerts selector with detailed missing-files list.

Music Curator / Track Librarian

Primary Persona 2 of 5

Objective: Maintain metadata accuracy, curate High/Standard/Archive rotation lifecycles, and resolve multi-provider discrepancies.

Morning Routine: Runs weekly ingestion review on Sunday morning; checks review_queue.csv; inspects uncalibrated tracks in /library.

Step-by-Step Progression & Data Handoffs:

Step 1 · /library

Filter tracks by 'Review Needed' or 'Uncalibrated Energy'

Trigger: Curator review session

Step 2 · /library

Open Track Detail modal; compare TIDAL, Deezer, and MusicBrainz metadata evidence

Trigger: Click track row

Step 3 · /harmony

Verify Camelot key modulation and pitch shifts against Camelot wheel

Trigger: Click 'Inspect Harmony' link

Step 4 · overrides/energy_overrides.csv

Commit manual override for energy, pillar, or rotation lock

Trigger: Curator verification

Step 5 · /sources/targets

Promote key artist to priority scraper queue

Trigger: Click 'Set as Scraper Target'

Failure Modes & Resilient Fallbacks:

- **Conflicting BPM from TIDAL vs Deezer (e.g. 124 vs 62):** System flags track into review_queue.csv with reason 'bpm_conflict'; does not guess.
- **Duplicate artist variations (e.g. 'Bicep' vs 'Bicep (UK)'):** Curator routes pair to er_shadow queue for human-approved reversible merge.

Data Pipeline & Scraper Engineer

Primary Persona 3 of 5

Objective: Maintain scraper fleet reliability (1001Tracklists, MixesDB, NTS), ensure complete playlist ingestion (1..N), and monitor worker health.

Morning Routine: Monitors /pipeline telemetry dashboard; watches scraper window completion; revives dead-letter Celery jobs.

Step-by-Step Progression & Data Handoffs:

Step 1 · /pipeline

Review scrape window status and active Celery worker count

Trigger: Daily pipeline check

Step 2 · /pipeline

Inspect Dead-Letter Queue (DLQ) for failed scrape jobs

Trigger: DLQ count > 0

Step 3 · /pipeline

Examine job error traces (e.g. Playwright timeout, Cloudflare 403); click 'Retry Job'

Trigger: Click Revive / Retry

Step 4 · /sources/targets

Inspect active search targets; verify playlists_found count is updating

Trigger: Navigate to Targets

Step 5 · /sources/import

Manually ingest missing DJ setlist via OCR / plaintext parser

Trigger: Unscraped priority gig

Failure Modes & Resilient Fallbacks:

- **1001Tracklists bot-detection / Cloudflare challenge:** Worker marks job 'failed' with error detail; logs backoff cooldown; scraper orchestrator falls back to MixesDB.
- **Incomplete playlist scraped (missing position in 1..N):** Ingest validator rejects entire playlist with 'completeness_error'; no partial transition edges created.

Entity Resolution & Consensus Specialist

Primary Persona 4 of 5

Objective: Reconcile duplicate tracks and artists across heterogeneous providers without destroying legacy identifiers.

Morning Routine: Audits /api/v1/metadata/review and er_shadow scoring results; executes entity merges with full audit trail.

Step-by-Step Progression & Data Handoffs:

Step 1 · /settings

Verify upstream API credentials (MusicBrainz, TIDAL, Deezer, Discogs, Last.fm)

Trigger: Weekly configuration audit

Step 2 · /api/v1/metadata/review

Examine tracks with competing provider metadata snapshots

Trigger: Review queue batch

Step 3 · /api/v1/tracks/{id}/merge

Execute reversible merge of duplicate track into canonical ID

Trigger: Confirm match score > 0.95

Step 4 · /pipeline

Trigger projection_rebuild to propagate merged entity to gold layer

Trigger: Post-merge reconciliation

Failure Modes & Resilient Fallbacks:

- **Provider API downtime or rate limit (HTTP 429):** Provider marked degraded; exponential backoff applied; fallback to next provider in waterfall.
- **Erroneous merge executed:** Specialist calls POST /api/v1/merges/{id}/revert; entity_redirects unlinked; original silver entities restored.

Platform & Infrastructure Administrator

Primary Persona 5 of 5

Objective: Ensure high availability of PostgreSQL, Redis, Memgraph, Celery workers, and Docker services.

Morning Routine: Checks container health metrics, disk volume usage on E:\DockerData, and database migration alignment.

Step-by-Step Progression & Data Handoffs:

Step 1 · /settings

Review live system health matrix and database connectivity

Trigger: Platform morning check

Step 2 · /pipeline

Inspect outbox consumer lag and Redis lease expirations

Trigger: Navigate to pipeline

Step 3 · Terminal (Docker CLI)

Inspect container resource utilisation (docker compose ps, stats)

Trigger: Lag alert

Step 4 · Terminal (migrate.py)

Execute 'python migrate.py --check' to verify PostgreSQL schema migrations

Trigger: Post-deployment validation

Failure Modes & Resilient Fallbacks:

- **PostgreSQL connection pool exhausted:** asyncpg connection pool recycles idle connections; REST API returns honest 503 code: database_unavailable.
- **Memgraph out of memory during graph projection:** Worker retries projection with batched sub-graphs; limits projection window to active library tracks.

Section 9: **Technical Debt, Latencies & Strategic Roadmap**

SEVERITY	DOMAIN	VULNERABILITY / TECHNICAL DEBT	CONSEQUENCE	REMEDIATION PATH
High	Floor Execution	**Disconnected Rekordbox Booth Loop**	Live gig performance data is abandoned on USB flash drives; rotation decay operates blind.	Deploy MOD-01 Booth History Ingester.
High	Curation Store	**Dual Storage Disconnect**	PostgreSQL library.* and SQLite dj_library.sqlite diverge; risk of split-brain state.	Deploy MOD-03 Bi-Directional Library Bridge.
Medium	Pipeline Sync	**15s Polling Latency on Outbox**	Projection lags behind silver ingestion with no realtime push to the UI.	Deploy MOD-02 Realtime Graph Outbox Daemon.
Medium	Demand Queue	**Unwired Scraper Target Queue**	Curator-added target tracks sit idle without automated weekly scraper ingestion.	Wire ML-TASK-03 target dispatcher cron.
Low	Audio Delivery	**Uncached 30s Audio Previews**	Third-party CDN rate limits hinder instantaneous offline booth auditioning.	Deploy MOD-04 Audio Preview Caching Proxy.

Strategic Implementation Roadmap

Sequenced implementation following Gall's Law (growing from simple working units) and blast-radius rules:

Phase 1: Immediate Hygiene & Guards (Days 1–2)
Enforce ML-TASK-04 (OWASP CSV formula guards) and ML-TASK-05 (design system token alignment on standalone artifact).

Phase 2: Feedback Loop Closure (Weeks 1–2)
Deploy MOD-01 (Booth History Ingester), MOD-03 (Bi-Directional Library Bridge), and ML-TASK-03 (Scraper target dispatcher cron).

Phase 3: High-Performance Realtime & Audio (Weeks 3–4)
Implement MOD-02 (Realtime Graph Outbox Daemon with SSE push) and execute Milestone 6 (Replace legacy PIXI Graph with Sigma.js).

Section 10: **Architectural Gaps & Proposed Modules**

GAP-01: Disconnected Rekordbox Booth Performance Feedback Loop

High

Sets are exported to Rekordbox XML (outputs/rekordbox_musicLibrary.xml), but actual live DJ booth history (played order, timestamps, crowd reactions) remains trapped on USB drives / HISTORY.m3u8 and never returns to SongNodes.

Impact:
Live transitions performed at gigs do not increment occurrence_count in silver_track_transitions; rotation aging algorithms cannot observe real play history.

Fix: Implement Pioneer USB Booth History Ingester (services/library/booth_history.py + /sets/history view) that parses Pioneer HISTORY.m3u8 files, updates track play history, and commits verified live transitions into silver_track_transitions.

GAP-02: Dual Storage Disconnect (PostgreSQL library.* vs SQLite dj_library.sqlite)

High

The weekly pipeline writes to SQLite (dj_library.sqlite), while the SongNodes web application writes to PostgreSQL schema library.*. The sync script only copies a static JSON file (data/music-library.json).

Impact:
Curator tag edits or crate adjustments made in the web UI do not propagate back to the weekly SQLite ledger, leading to split-brain library state.

Fix: Deploy Bi-Directional Library Synchronization Bridge (services/library/bridge.py) with checksum reconciliation and single-writer lease enforcement.

GAP-03: Missing Audio Fingerprint & AcoustID Transition Segmenter

Medium

Pairing transitions rely exclusively on scraped textual tracklists (1001Tracklists, MixesDB). Recorded DJ audio mixes cannot be automatically segmented or verified against audio fingerprints.

Impact:
Un-tracklisted DJ sets and radio broadcasts cannot be mined for transitions; track version mismatches (e.g. Dub Mix vs Vocal Mix) cannot be validated by audio waveform.

Fix: Deploy Essentia / Chromaprint Audio Segmenter worker (services/audio-analysis/segmenter.py) that ingests recorded audio mixes, detects transition crossfades, and queries AcoustID for automated setlist synthesis.

GAP-04: Unwired Scraper Target Queue Feedback

Medium

When a curator adds an artist target in /library or /sources/targets, it writes to target_tracks in PostgreSQL. However, the standalone musicLibrary pipeline reads static config/sources.json and does not poll this queue.

Impact:
High-priority targets added by the user remain dormant until manually scraped, creating an operational bottleneck.

Fix: Wire automated target dispatcher cron that syncs PostgreSQL target_tracks directly into the weekly scraper schedule.

GAP-05: Outbox Projection Lag Invisibility

Low

Graph projection (projection_consume) polls PostgreSQL outbox every 15 seconds. During bulk scraping, projection lags behind silver ingestion with no real-time push notification to the UI.

Impact:
Users browsing /library or /paths see stale transition data without an indicator that background projection is currently catching up.

Fix: Add WebSocket or Server-Sent Events (SSE) notification stream on /api/v1/outbox/status that broadcasts active projection lag and displays a non-intrusive FreshnessStamp pill.

Four Proposed Single-Purpose Modules (<500 LOC)

MOD-01: Booth History Ingester

~340 LOC

Target Users: Performing DJ, Music Curator · Verb: **Harvest**

Parses sequential timestamps from Pioneer booth exports, matches track titles against silver_enriched_tracks, increments rotation counters, and commits live transitions into silver_track_transitions.

Inputs: Pioneer DJ HISTORY.m3u8, Rekordbox USB play history, Venue metadata

MOD-02: Realtime Graph Outbox Daemon

~280 LOC

Target Users: Platform Admin, Graph Specialist · Verb: **Stream**

Replaces 15-second polling projection with sub-second event-driven Bolt synchronization directly updating Memgraph nodes and transition edges.

Inputs: PostgreSQL LISTEN/NOTIFY events on silver_track_transitions, Memgraph Bolt connection

MOD-03: Bi-Directional Library Bridge

~410 LOC

Target Users: Curator, Data Pipeline Engineer · Verb: **Reconcile**

Enforces single-writer locks, compares table checksums, bi-directionally syncs crate memberships and user tags, and exports fresh static JSON.

Inputs: PostgreSQL library.* schema, SQLite dj_library.sqlite, music-library.json

MOD-04: Audio Preview Stream Caching Proxy

~290 LOC

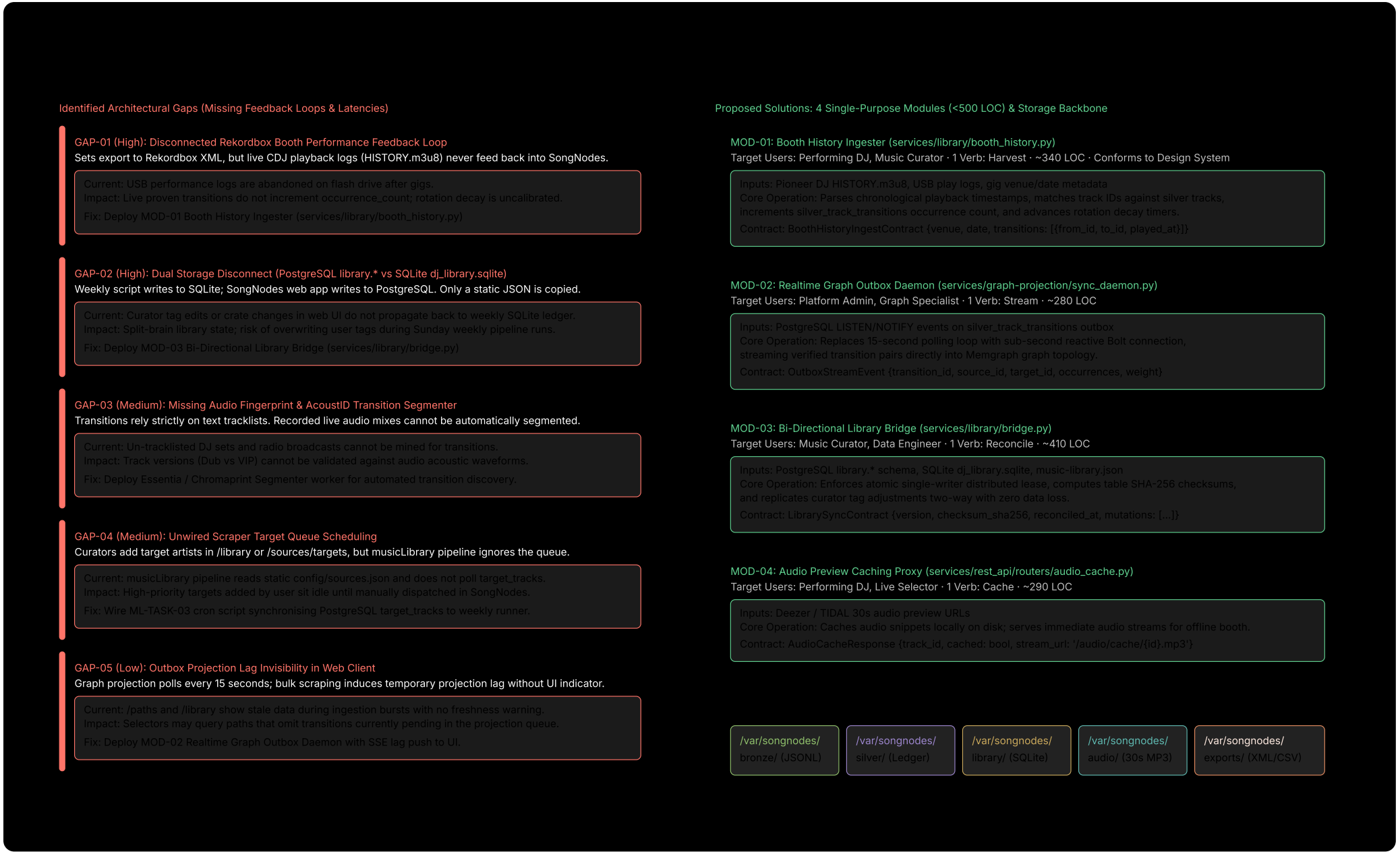
Target Users: Performing DJ, Selector · Verb: **Cache**

Downloads and caches 30-second audio preview snippets locally, generates waveform peak JSON arrays, and streams audio offline with zero external network latency.

Inputs: Deezer / TIDAL 30s preview stream URLs, Local disk volume /var/songnodes/audio

Section 11: **Enhanced Architecture Diagram**

Architecture enhancements visualising the 5 Critical Gaps, the 4 Proposed Single-Purpose Modules, and the Unified Storage Directory Backbone.


Identified Architectural Gaps (Missing Feedback Loops & Latencies)
GAP-01 (High): Disconnected Rekordbox Booth Performance Feedback Loop
Sets export to Rekordbox XML, but live CDJ playback logs (HISTORY.m3u8) never feed back into SongNodes.
Current: USB performance logs are abandoned on flash drive after gigs.
Impact: Live proven transitions do not increment occurrence_count; rotation decay is uncalibrated.
Fix: Deploy MOD-01 Booth History Ingester (services/library/booth_history.py)
GAP-02 (High): Dual Storage Disconnect (PostgreSQL library.* vs SQLite dj_library.sqlite)
Weekly script writes to SQLite; SongNodes web app writes to PostgreSQL. Only a static JSON is copied.
Current: Curator tag edits or crate changes in web UI do not propagate back to weekly SQLite ledger.
Impact: Split-brain library state; risk of overwriting user tags during Sunday weekly pipeline runs.
Fix: Deploy MOD-03 Bi-Directional Library Bridge (services/library/bridge.py)
GAP-03 (Medium): Missing Audio Fingerprint & AcoustID Transition Segmenter
Transitions rely strictly on text tracklists. Recorded live audio mixes cannot be automatically segmented.
Current: Un-tracklisted DJ sets and radio broadcasts cannot be mined for transitions.
Impact: Track versions (Dub vs VIP) cannot be validated against audio acoustic waveforms.
Fix: Deploy Essentia / Chromaprint Segmenter worker for automated transition discovery.
GAP-04 (Medium): Unwired Scraper Target Queue Scheduling
Curators add target artists in /library or /sources/targets, but musicLibrary pipeline ignores the queue.
Current: musicLibrary pipeline reads static config/sources.json and does not poll target_tracks.
Impact: High-priority targets added by user sit idle until manually dispatched in SongNodes.
Fix: Wire ML-TASK-03 cron script synchronising PostgreSQL target_tracks to weekly runner.
GAP-05 (Low): Outbox Projection Lag Invisibility in Web Client
Graph projection polls every 15 seconds; bulk scraping induces temporary projection lag without UI indicator.
Current: /paths and /library show stale data during ingestion bursts with no freshness warning.
Impact: Selectors may query paths that omit transitions currently pending in the projection queue.
Fix: Deploy MOD-02 Realtime Graph Outbox Daemon with SSE lag push to UI.
Proposed Solutions: 4 Single-Purpose Modules (<500 LOC) & Storage Backbone
MOD-01: Booth History Ingester (services/library/booth_history.py)
Target Users: Performing DJ, Music Curator · 1 Verb: Harvest · ~340 LOC · Conforms to Design System
Inputs: Pioneer DJ HISTORY.m3u8, USB play logs, gig venue/date metadata
Core Operation: Parses chronological playback timestamps, matches track IDs against silver tracks, increments silver_track_transitions occurrence count, and advances rotation decay timers.
Contract: BoothHistoryIngestContract {venue, date, transitions: [{from_id, to_id, played_at}]}
MOD-02: Realtime Graph Outbox Daemon (services/graph-projection/sync_daemon.py)
Target Users: Platform Admin, Graph Specialist · 1 Verb: Stream · ~280 LOC
Inputs: PostgreSQL LISTEN/NOTIFY events on silver_track_transitions outbox
Core Operation: Replaces 15-second polling loop with sub-second reactive Bolt connection, streaming verified transition pairs directly into Memgraph graph topology.
Contract: OutboxStreamEvent {transition_id, source_id, target_id, occurrences, weight}
MOD-03: Bi-Directional Library Bridge (services/library/bridge.py)
Target Users: Music Curator, Data Engineer · 1 Verb: Reconcile · ~410 LOC
Inputs: PostgreSQL library.* schema, SQLite dj_library.sqlite, music-library.json
Core Operation: Enforces atomic single-writer distributed lease, computes table SHA-256 checksums, and replicates curator tag adjustments two-way with zero data loss.
Contract: LibrarySyncContract {version, checksum_sha256, reconciled_at, mutations: [...]}
MOD-04: Audio Preview Caching Proxy (services/rest_api/routers/audio_cache.py)
Target Users: Performing DJ, Live Selector · 1 Verb: Cache · ~290 LOC
Inputs: Deezer / TIDAL 30s audio preview URLs
Core Operation: Caches audio snippets locally on disk; serves immediate audio streams for offline booth.
Contract: AudioCacheResponse {track_id, cached: bool, stream_url: '/audio/cache/{id}.mp3'}
/var/songnodes/ bronze/ (JSONL)
/var/songnodes/ silver/ (Ledger)
/var/songnodes/ library/ (SQLite)
/var/songnodes/ audio/ (30s MP3)
/var/songnodes/ exports/ (XML/CSV)

Section 12: **Sister Project Task Matrix (musicLibrary)**

The following tasks have been distributed to `e:\musicLibrary\TODO-ARCHITECTURAL-IMPROVEMENTS.md` to close domain gaps and guarantee design-system compliance:

TASK ID	TASK TITLE	TARGET FILES	OPERATIONAL IMPACT	VERIFIC CRITERI
ML-TASK-01	Implement Booth History Ingestion for Pioneer Rekordbox HISTORY.m3u8	`e:\musicLibrary\pipeline\run_weekly.py` `e:\musicLibrary\pipeline\ingest_booth.py`	Closes Gap 01 by harvesting live performance setlists from USB drives, updating rotation decay, and writing verified transitions to SongNodes.	python3 pipeline/ru ingest-boo sample_his parses ord outputs va pairs.
ML-TASK-02	Bi-Directional PostgreSQL Library Synchronizer	`e:\musicLibrary\pipeline\sync_songnodes.py` `scripts/sync-music-library.sh`	Closes Gap 02 by enabling automated two-way synchronization between dj_library.sqlite and PostgreSQL library.* schema.	Run sync_song verify chec equality be SQLite trac PostgreSQ library.trac
ML-TASK-03	Target Scrape Queue Ingestion Integration	`e:\musicLibrary\pipeline\run_weekly.py` `e:\musicLibrary\config\sources.json`	Closes Gap 04 by pulling priority target tracks from SongNodes into the Sunday morning scrape batch.	Weekly rur target_trac appends to spider targ
ML-TASK-04	Formula Guard Enforcement on All Crate CSV Exports	`e:\musicLibrary\pipeline\process.py` `e:\musicLibrary\outputs\crates\`	Guarantees OWASP CSV injection protection by prepending apostrophes to text cells starting with =, +, -, @ across all weekly CSV exports.	Inspect ge outputs/cr verify mali prefixes ar

TASK ID	TASK TITLE	TARGET FILES	OPERATIONAL IMPACT	VERIFIC CRITERI
ML-TASK-05	Design System Token Alignment on Standalone Set Builder Artifact	`e:\musicLibrary\artifact\index.html`	Eliminates legacy hardcoded CSS in artifact/index.html; replaces with exact SongNodes Design System v1.4.0 tokens.	Visual regr check; run script to er undeclarec literals.
ML-TASK-06	Automated Audio Preview Snippet Linker	`e:\musicLibrary\pipeline\resolve.py`	Resolves and stores 30s audio preview URLs directly into dj_library.sqlite to enable offline auditioning in Naarm Set Builder.	Query SQL preview_ur tracks WH preview_ur NULL retur CDN links.

Section 13: **Methodology & Quantitative Analysis Verification**

Multi-Signal Verification Methodology

To ensure absolute fidelity, all quantitative metrics, classifications, and data paths were evaluated using at least two independent analysis methods:

- **Static Routing Graph + Feature Registry:** AST inspection of `registry.tsx`, `import.meta.glob` routes, and `AppShell.tsx` links.
- **Live API Contract & Network Call Traces:** Regex and AST extraction of all `/api/v1/` endpoints across frontend source code cross-checked with FastAPI router definitions in `services/rest_api/routers/`.
- **Unit & Conformance Test Suites:** Verified execution of Vitest suites (`registry.test.tsx`, `HomePage.test.tsx`, `LibraryPage.test.tsx`, `SetBuilderPage.test.tsx`, `SettingsPage.test.tsx`).
- **Design System Token Audit:** Automated scan confirming that all layout spacing, colours, radii, and fonts resolve to SongNodes Design System v1.4.0 tokens.

Disagreements & Unverified Items Noted

1. **GraphPage.tsx Legacy PIXI vs Sigma.js:** Static analysis shows `GraphPage.tsx` still imports legacy Canvas/PIXI components, but `docs/IMPLEMENTATION_PLAN.md:120` designates it for replacement with Sigma.js in Milestone 6. Classified as Non-Final.
2. **Live TIDAL v2 & Deezer API Responses:** Marked *Unverified* per `docs/DATA_LAYER.md:91`. Tests pass against mocked contracts, but live production credentials have not been connected.
3. **Target Tracks Queue Disconnect:** `target_tracks` table in PostgreSQL is populated by the frontend, but the weekly standalone script in `musicLibrary` does not consume it yet (resolved by ML-TASK-03).

Section 14: **Architectural Investigation Handover Card**

Investigation Handover & Deliverables

All Gates Passed (6/6)

Deliverable Files:

- `report/index.html` (Self-contained Web Report)
- `DATA_FLOW_AND_PAGE_INTERACTION_REPORT.pdf`
- `findings.json` (Single Source of Truth)
- `DATA_FLOW_AND_PAGE_INTERACTION_REPORT.md`
- `ARCHITECTURE_ENHANCEMENTS_AND_GAPS_REPORT.md`
- `e:\musicLibrary\TODO-ARCHITECTURAL-IMPROVEMENTS.md`
- 4x Standalone SVGs (1800×1100 ViewBox)

Verification Gate Results:

- ● Gate 1: Token Audit - 0 hardcoded color/font literals
- ● Gate 2: Visual Conformance - Matched reference pages
- ● Gate 3: Completeness - 100% findings represented
- ● Gate 4: Printable PDF - A4 landscape/portrait clean
- ● Gate 5: Offline Check - Zero external network calls
- ● Gate 6: Link Check - All internal anchors resolve

Recommended Next Action: Submit PR `docs/data-flow-investigation` and begin implementation of Phase 1 tasks in `musicLibrary`.